Sahaay Club: Automatic Waste Segregator

Nishitha D (EE23B188)

Overview

I contributed to the software module of the Automatic Waste Segregator, a real-time multi-waste segregation bin that could detect up to 5 categories of waste and segregate them into their respective classes. This was an effort to improve recycling efficiency and waste management.

The goal of the project was to develop a bin that could identify the waste kept on top of it with a camera, identify the class of the object, and rotate the lid to the corresponding section to drop the item.

Some Results of the Project:

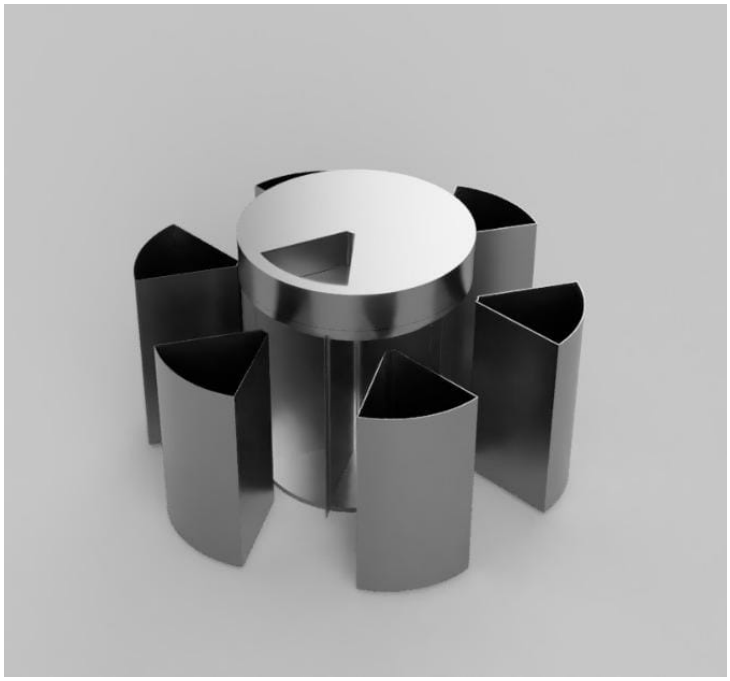

Figure 1: Visualised Design

Figure 2: In-process Mechanical Module

Project Resources

The final results are at: **Drive Link**.

The complete source code and dataset are available at: **GitHub Repository**.

Computer Vision Approach

For the computer vision aspect, we explored three different architectures and used transfer learning along with our own custom dataset of 5000 images.

The chosen classes were: **Plastic waste, Paper, Mixed (E-waste and food waste), Metal, and Glass**.

Dataset Challenges

Initially, the Kaggle dataset we found contained only 200 images per class. This would easily lead to overfitting due to limited variation in camera angles and lighting conditions.

Hence, we collected 1000 samples per class under varied conditions using both personal captures and online sources.

Model Training

We fine-tuned ResNet, MobileNet, and InceptionNet using TensorFlow Keras data augmentation.

```
data_augmentation = tf.keras.Sequential([
  layers.RandomFlip("horizontal_and_vertical"),
  layers.RandomRotation(0.2),
  layers.RandomZoom(0.2),
])
```

Initial Model

A simple CNN model was first implemented:

```
Layers = [
  Conv2D(72, kernel_size=(3,3), input_shape=(256,256,3)),
  Conv2D(72, kernel_size=(3,3), activation='relu'),
  MaxPooling2D(pool_size=(2,2)),
  Dropout(0.5),
  Flatten(),
  Dense(250, activation='relu'),
  Dense(5, activation='softmax')
]
model = keras.Sequential(Layers)
```

This achieved an accuracy of 66%, as it had only one convolution block and was underfitting.

ResNet-34

ResNet-34 was chosen to fine-tune, but due to missing Keras pre-trained weights, it was implemented manually following the original paper. However, the model underfit due to confusion between classes (especially glass and plastic).

Results (ResNet-34)

```
Epoch 30: val_accuracy did not improve from 0.62315
57/57 [==============================] - 34s 592ms/step
```

```
- loss: 1.0155 - accuracy: 0.6286
- val_loss: 1.0400 - val_accuracy: 0.6232
```

InceptionNetV3

Initially achieved 63% validation accuracy, later improved to 86.7% with more epochs, trainable layers, and data augmentation.

```
Epoch 30: val_accuracy did not improve from 0.93528
128/128 [==============================] - 100s 785ms/step
- loss: 0.0443 - accuracy: 0.9866
- val_loss: 0.3678 - val_accuracy: 0.9202
```

MobileNetV2

```
Train Accuracy: 0.9447
Validation Accuracy: 0.9884

Validation F1 Score (macro): 0.9797
```

Confusion Matrix

```
[[674   1   2   0   3   0]
 [ 13 528   3   1   0   0]
 [ 13   1 519   3   2   0]
 [ 12   0   3 878   2   0]
 [ 10   1   4   1 479   0]
 [  0   0   0   0   0 576]]
```

Classification Report (Summary)

$$\text{Accuracy} = 0.98$$
$$\text{Macro Average F1} = 0.98$$
$$\text{Weighted Average F1} = 0.98$$

Conclusion

The MobileNetV2 model emerged as the best performer. Reasons:

- Smaller model size $\Rightarrow$ better generalization.
- Reduced overfitting due to fewer parameters.

- No fully connected layer (uses global average pooling).

This project provided a deep understanding of:

- Transfer learning techniques.
- Implementing CNN architectures from scratch.
- Fine-tuning high-performance models for real-world datasets.

We decided to go with MobileNetV2 as:

- Its lightweight architecture allows for efficient deployment on mobile and embedded devices with limited computational resources
- High accuracy achieved on the validation and test dataset
- The model's small size enables faster inference times, making it suitable for real-time applications.